Participating in the Monitoring Process

Active Record controls the life cycle of model objects—it creates them, monitors them as they're modified, saves and updates them, and watches sadly as they're destroyed. Using callbacks, Active Record lets our code participate in this monitoring process. We can write code that gets invoked at any significant event in the life of an object. With these callbacks we can perform complex validation, map column values as they pass in and out of the database, and even prevent certain operations from completing.

Active Record defines sixteen callbacks. Fourteen of these form before-after pairs and bracket some operation on an Active Record object. For example, the `before_destroy` callback will be invoked just before the `destroy` method is called, and `after_destroy` will be invoked after. The two exceptions are `after_find` and `after_initialize`, which have no corresponding `before_`*xxx* callback. (These two callbacks are different in other ways too, as we'll see later.)

In the following figure we can see how Rails wraps the fourteen paired callbacks around the basic create, update, and destroy operations on model objects. Perhaps surprisingly, the before and after validation calls aren't strictly nested.